

Skill-05

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1000
#define MAX_TOKENS 100
#define MAX_TOKEN 500

typedef enum {
    WORD,
    SINGLE_QUOTED,
    DOUBLE_QUOTED
} TokenType;

typedef struct {
    char value[MAX_TOKEN];
    TokenType type;
} Token;

Token tokens[MAX_TOKENS];
int tokenCount = 0;

void addToken(char *value, TokenType type) {
    if (tokenCount >= MAX_TOKENS)
        return;

    strcpy(tokens[tokenCount].value, value);
    tokens[tokenCount].type = type;
    tokenCount++;
}

void expandVariable(char *input, char *output) {
    int i = 0, j = 0;

    while (input[i] != '\0') {
        if (input[i] == '$') {
            i++;

            char name[100];
            int k = 0;

            while (isalnum(input[i]) || input[i] == '_' || input[i] == '-') {
                name[k++] = input[i++];
            }

            name[k] = '\0';

            char *value = getenv(name);
```

```

        if (value != NULL) {
            for (int x = 0; value[x] != '\0'; x++)
                output[j++] = value[x];
        }
        else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int tokenize(char *input) {
    int i = 0;

    while (input[i] != '\0') {
        while (isspace(input[i]))
            i++;

        if (input[i] == '\0')
            break;

        char token[MAX_TOKEN];
        int j = 0;
        int hasSingle = 0;
        int hasDouble = 0;

        while (input[i] != '\0' && !isspace(input[i])) {
            if (input[i] == '"') {
                hasSingle = 1;
                i++;

                while (input[i] != '\0' && input[i] != '"') {
                    token[j++] = input[i++];
                }

                if (input[i] != '"') {
                    printf("Syntax error: unclosed single quote\n");
                    return 0;
                }
            }

            i++;
        }
        else if (input[i] == "'") {
            hasDouble = 1;
            i++;

            char temp[MAX_TOKEN];
            int k = 0;

            while (input[i] != '\0' && input[i] != "'") {
                temp[k++] = input[i++];
            }

            if (input[i] != "'") {
                printf("Syntax error: unclosed double quote\n");
                return 0;
            }
        }
    }
}

```

```

    }

    temp[k] = '\0';

    char expanded[MAX_TOKEN];
    expandVariable(temp, expanded);

    for (int x = 0; expanded[x] != '\0'; x++)
        token[j++] = expanded[x];

    i++;
}
else {
    token[j++] = input[i++];
}
}

token[j] = '\0';

if (j > 0) {
    if (hasSingle)
        addToken(token, SINGLE_QUOTED);
    else if (hasDouble)
        addToken(token, DOUBLE_QUOTED);
    else
        addToken(token, WORD);
}
}

return 1;
}

void printTokens() {
    printf("\nParsed Tokens:\n");

    for (int i = 0; i < tokenCount; i++) {
        printf("Token %d: %s", i + 1, tokens[i].value);

        if (tokens[i].type == SINGLE_QUOTED)
            printf(" [Single Quoted]");
        else if (tokens[i].type == DOUBLE_QUOTED)
            printf(" [Double Quoted]");
        else
            printf(" [Word]");

        printf("\n");
    }
}

int main() {
    char input[MAX_INPUT];

    printf("Quote Parser\n");
    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

```

```

input[strcspn(input, "\n")] = '\0';

tokenCount = 0;

if (tokenize(input)) {
    printTokens();
    printf("\nParsing successful\n");
}

return 0;
}

```

Screen Shots:

```

sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ gcc Skill_5.c -o Skill_5
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ ./Skill_5
Quote Parser
Enter command: echo 'Hello World'

Parsed Tokens:
Token 1: echo [Word]
Token 2: Hello World [Single Quoted]

Parsing successful
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ gcc Skill_5.c -o Skill_5
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ ./Skill_5
Quote Parser
Enter command: echo "Hello $USER"

Parsed Tokens:
Token 1: echo [Word]
Token 2: Hello sohan_jadala [Double Quoted]

Parsing successful
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ |

```